VIT Bhopal University

School of Computer Science and Engineering

Open Source Software | NGMC Course | AY 2025-26

The Open Source Audit

A Capstone Project Report

Chosen Software: Git — Distributed Version Control System (GPL v2)

Student Name	Naba Krishna Hazarika
Registration No.	24MIP10020
Email	naba.24mip10020@vitbhopal.ac.in
Course	Open Source Software (NGMC)
Submission Date	29th March 2026

VITyarthi Platform Submission | March 2026

Abstract

This report is a structured audit of Git, the world's most widely used version control system. It covers the origin story behind Git's creation, what the GPL v2 licence actually means for users and companies, the ethical questions that open source raises, how Git is installed and managed on a Linux system, the ecosystem it has enabled, and an honest comparison against Perforce Helix Core, its closest proprietary alternative. Five shell scripts demonstrating practical Linux skills are included with full source code, screenshot placeholders, and explanations of the concepts each script uses.

1. Origin and Philosophy

1.1 The Problem Git Was Created to Solve

Before Git, teams mostly used CVS or Subversion. Both needed one central server. If the server was slow or down, nobody could work. This was okay for small teams, but a big problem for Linux with thousands of developers worldwide.

In 2002 the Linux team started using BitKeeper (free from BitMover). It was good, but not open-source. In 2005 a developer tried to study how it worked. BitMover got angry, called it a license violation, and stopped access for everyone the next day.

Linus Torvalds decided: never again depend on someone else's tool they can take away. Instead of asking or buying something else, he wrote Git himself. In about 10 days he had a working version.

Git was designed to be:

- very fast
- fully distributed (everyone has a full copy)
- cryptographically secure (history can't be secretly changed)

Linus later said his rule was simple: "do the opposite of CVS".

Within one year most big open-source projects switched to Git. Today over 93% of professional developers use it, and GitHub (built on Git) hosts hundreds of millions of projects.

What started as one person's anger became the foundation of modern software development.

1.2 The Licence — What GPL v2 Actually Says

The Git source code lives at https://github.com/git/git. At the root of the repository is a file called COPYING containing the full text of the GNU General Public Licence Version 2 (GPL v2) — the same licence Torvalds chose for the Linux kernel. Reading this file is not optional; it tells you exactly what you are and are not allowed to do.

Richard Stallman defined four freedoms that make software genuinely free:

1. Freedom 0: Run the software for any purpose without paying or asking permission.
2. Freedom 1: Study how it works and modify it. This requires access to the source code.
3. Freedom 2: Redistribute copies freely so others can benefit.

4. Freedom 3: Distribute your modified versions. GPL v2's key condition: if you do this, you must release those modifications under GPL v2 too. You cannot take the code, improve it, and sell a closed-source version.

Git grants all four freedoms. The critical clause is the copyleft requirement in Freedom 3. This is the core difference between GPL and a permissive licence like MIT. MIT says do whatever you want, including closing the source. GPL v2 says you can do almost anything, but if you distribute a derivative work, it must stay open.

Can a company modify Git and sell it without sharing changes? Not if they distribute it. If a company ships a modified Git binary to customers, they must provide the source. But if they only run a modified Git internally without distributing it, GPL v2 does not compel them to share. This is why GitHub can build a proprietary platform on top of Git — they are running a service, not distributing a modified binary.

GPL v2 vs MIT: For a personal utility I would probably choose MIT because I want maximum adoption without friction. For infrastructure that a whole community depends on, I would choose GPL v2. It is the strongest guarantee that the software cannot be taken away from the people who rely on it — which is precisely the lesson Torvalds learned from BitKeeper.

Free as in beer vs free as in freedom: "Free beer" means zero cost. "Free as in freedom" (or libre in French) means the right to use, study, change, and share. BitKeeper was free as in beer but not free as in freedom — and that distinction cost the entire Linux kernel team their workflow. Git, under GPL v2, is free in both senses, but the second kind of freedom is the one that matters.

1.3 The Ethics of Open Source

Should all software be open source? Not necessarily. Open source works best when it aligns with how a community wants to share knowledge. The case for it is strong: inspectable code is more secure, more likely to last, and more adaptable than a black box. But some software requires enormous sustained investment — medical devices, aerospace systems, specialised industrial tools — and the volunteer model does not always support that. A more useful question is whether shared infrastructure should be open. My view is yes: tools that a whole community depends on and that give any single vendor dangerous leverage if closed should be open. Git is a perfect example.

Is it ethical to build a profitable business on free community labour? GitHub was acquired by Microsoft for $7.5 billion. The Git project itself, maintained by volunteers under the Software Freedom Conservancy, received nothing from that. There is a credible defence: GitHub genuinely added value — the web interface, pull requests, Actions — and companies that add value can capture some of it. But the ethical obligation scales with the benefit. A startup using Git perhaps owes a good bug report. A company generating billions on infrastructure maintained by unpaid contributors owes those contributors meaningful financial support. The industry does not consistently provide this.

Standing on the shoulders of giants: Every system you will build in your career rests on open-source foundations. Git itself is proof that open source enables rather than undermines original innovation: Torvalds did not re-write a compression library or a cryptographic hash function from scratch. He built on open tools and focused entirely on the new problem he was solving. The result is something that has itself become the foundation that millions of other projects stand on.

2. Linux Footprint

Installation: On Debian/Ubuntu: sudo apt install git. On Fedora/RHEL: sudo dnf install git. For the latest version, it can be compiled from source using gcc, make, libssl-dev, libcurl4-openssl-dev, and zlib1g-dev — all of which are themselves open-source tools.

Key directories after installation:

Path	Contents
/usr/bin/git	Main Git executable
/usr/lib/git-core/	Sub-commands: git-fetch, git-merge, git-rebase, etc.
/usr/share/git-core/	Templates and helper scripts
/usr/share/doc/git/	Documentation and copyright notices
~/.gitconfig	Per-user configuration (name, email, aliases)
/etc/gitconfig	System-wide configuration
.git/ (in any repo)	Full repository: object store, refs, HEAD, config

User and group: Git does not run as a dedicated system user during normal use — it runs as whoever invokes it. On a shared server, a dedicated git user is created (sudo adduser --system git) so the service runs with minimal privileges. This is the principle of least privilege: if the git service is compromised, it cannot touch files owned by other users or root.

Service management: Git itself is not a persistent daemon. The git daemon command provides a read-only server that can be run as a systemd service (systemctl start git-daemon), but most teams use SSH instead — meaning the service being managed is OpenSSH, not Git directly. This is a strength: Git integrates with existing system security infrastructure rather than requiring its own.

Update model: Security patches are coordinated through the Git project's security mailing list, CVE identifiers are assigned publicly, and distribution maintainers package updates within hours. Running sudo apt upgrade git is all that is needed on most systems. The entire patch and disclosure process is public and auditable — a direct benefit of the open-source model that no proprietary tool can match.

3. The FOSS Ecosystem

Dependencies: Git is written primarily in C, with some Perl and shell components. Its core dependencies are OpenSSL/libcrypto for hashing and HTTPS transport, libcurl for remote access, zlib for object compression, Perl for sub-commands like git-svn, and OpenSSH for authenticated push/pull. Every one of these is open source, which means the entire stack from operating system to version control is auditable and freely available.

What Git has enabled: GitHub (2008) added a web interface, pull requests, and social coding on top of Git and was acquired for $7.5 billion. GitLab (2011) is an open-source alternative that can be self-hosted — popular with organisations that need full control of their code. Gitea and Forgejo are lightweight self-hosted options; Forgejo itself is a community fork of Gitea, which is an illustration of the forking principle in action. libgit2 is a C library for embedding Git in other applications, used by GitHub, GitLab, and most GUI clients. GitHub Actions, GitLab CI, Jenkins, and CircleCI are all built around Git's branching and commit model: the concept of a pipeline triggered by a git push is now the standard unit of modern software delivery.

Git and the LAMP stack: Git does not sit inside LAMP (Linux, Apache, MySQL, PHP) as a runtime component, but it is inseparable from how LAMP applications are developed and deployed. Code is written in a Git repository, pushed to a remote, and a CI/CD pipeline deploys to the Apache/MySQL server — often using git pull on the server itself. Configuration files, database migrations, and server scripts are all tracked in Git. It is the connective tissue of the entire development workflow.

Community and governance: Git development happens on a public mailing list (git@vger.kernel.org) entirely in plain-text email — deliberately transparent and low-technology. The project is governed by the Software Freedom Conservancy, a non-profit that holds the Git trademark and assists with GPL enforcement. Junio C. Hamano has been the maintainer since 2005. No company owns Git, and no acquisition can relicense or shut it down.

4. Open Source vs Proprietary

The closest proprietary alternative to Git is Perforce Helix Core, widely used in game development and aerospace for large binary files and fine-grained file locking.

Dimension	Git (GPL v2)	Perforce Helix Core
Cost	Free. Hosting via GitHub/GitLab with free tiers.	Licence fees; enterprise pricing often tens of thousands per year.
Security	Full source is public. CVEs tracked openly. Anyone can audit the code.	Closed source. Security depends entirely on Perforce's internal team.
Support	Community support plus commercial options (GitHub Enterprise, GitLab EE).	Full commercial SLAs, but vendor controls pricing and end-of-life dates.
Freedom to modify	Complete. Any team can fork, patch, and redistribute.	None. Users cannot adapt the tool to their needs.
Community control	Governed by Software Freedom Conservancy. No single owner.	Governed by Perforce Software Inc. Roadmap set by corporate priorities.
Large binary files	Requires git-lfs. Can struggle with very large binary assets natively.	Native strength. Designed for multi-GB assets from the start.

Verdict: For almost any software development context, Git is the right choice. It is faster for branching and merging, its ecosystem is incomparably richer, its security model is stronger because the source is open, and it costs nothing. The only genuine case for Perforce is a team primarily working with very large binary assets where file locking matters more than distributed workflows. Would I contribute to Git? Yes — a realistic first step is a documentation fix or a small well-tested patch submitted to the mailing list.

5. Shell Scripts

All five scripts were written and tested on a Linux system. Each is submitted as a .sh file in the GitHub repository.

5.1 Script 1 — System Identity Report

```bash
#!/bin/bash
# ── Variables ──────────────────────────────────────────────
STUDENT_NAME="Naba Krishna Hazarika"   # Student's full name
REG_NO="24MIP10020"                    # Registration number
SOFTWARE_CHOICE="Git"                  # Chosen OSS project

# ── Gather system information using command substitution ────
KERNEL=$(uname -r)            # Kernel release version
ARCH=$(uname -m)              # Hardware architecture (x86_64 etc.)
USER_NAME=$(whoami)           # Name of the currently logged-in user
HOME_DIR=$HOME                # Home directory path from environment variable
HOSTNAME=$(hostname)          # Machine hostname
UPTIME=$(uptime -p)           # Human-readable uptime (e.g., "up 2 hours")
CURRENT_DATE=$(date '+%A, %d %B %Y')   # e.g., "Monday, 29 March 2026"
CURRENT_TIME=$(date '+%H:%M:%S')       # e.g., "11:30:00"

# ── Detect Linux distribution name ─────────────────────────
if [ -f /etc/os-release ]; then
    DISTRO=$(grep "^PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d '"')
else
    DISTRO="Unknown Linux Distribution"
fi
# ── Display formatted output ───────────────────────────────
echo "================================================================"
echo "           OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT          "
echo "================================================================"
echo ""
echo "  Student : $STUDENT_NAME ($REG_NO)"
echo "  Project : $SOFTWARE_CHOICE — Open Source Software Audit"
echo ""
echo "----------------------------------------------------------------"
echo "  SYSTEM INFORMATION"
echo "----------------------------------------------------------------"
```

```
echo "----------------------------------------------------------------"
echo "  SYSTEM INFORMATION"
echo "----------------------------------------------------------------"
echo "  Distribution : $DISTRO"
echo "  Kernel       : $KERNEL"
echo "  Architecture : $ARCH"
echo "  Hostname     : $HOSTNAME"
echo "  Logged in as : $USER_NAME"
echo "  Home Dir     : $HOME_DIR"
echo "  Uptime       : $UPTIME"
echo "  Date         : $CURRENT_DATE"
echo "  Time         : $CURRENT_TIME"
echo ""
echo "----------------------------------------------------------------"
echo "  LICENSE INFORMATION"
echo "----------------------------------------------------------------"
echo "  Linux Kernel License : GNU General Public License v2 (GPL v2)"
echo "  This means you have the freedom to:"
echo "    * Run the OS for any purpose"
echo "    * Study and modify the kernel source code"
echo "    * Redistribute copies freely"
echo "    * Share your modifications under GPL v2"
echo ""
echo "================================================================"
echo "  End of System Identity Report"
echo "================================================================"
```

Explanation: This script demonstrates variables, command substitution $(...), a conditional file check if [-f ...], and formatted output with echo. Commands like uname -r, whoami, and date are captured into variables and printed. Distribution detection reads /etc/os-release, the standard file for distro identification on modern Linux.

5.2 Script 2 — FOSS Package Inspector

```
#!/bin/bash
# ── Target package — change this to test other packages ──────
PACKAGE="git"   # Primary package being audited in this project

# ── Helper function: check package on apt/dpkg systems ──────
check_package_apt() {
    # dpkg -l lists installed packages; grep -q = quiet (just checks exit code)
    # ^ii means the package is fully installed in dpkg state
    if dpkg -l "$1" 2>/dev/null | grep -q "^ii"; then
        echo "[INSTALLED] $1 is present on this system."
        echo ""
        echo "  Package details:"
        # dpkg -s gives detailed status; grep -E filters multiple fields
        dpkg -s "$1" 2>/dev/null | grep -E "^(Version|Maintainer|Homepage|Description)"
    else
        echo "[NOT FOUND] $1 is not installed via apt/dpkg."
        echo "  To install: sudo apt install $1"
    fi
}

# ── Helper function: check package on rpm systems ────────────
check_package_rpm() {
    # rpm -q queries if package is installed (exits 0 if yes, non-zero if not)
    if rpm -q "$1" &>/dev/null; then
        echo "[INSTALLED] $1 is present on this system."
        echo ""
        echo "  Package details:"
        # rpm -qi gives detailed info; grep filters version, license, summary
        rpm -qi "$1" | grep -E "^(Version|License|Summary|URL)"
    else
        echo "[NOT FOUND] $1 is not installed via rpm."
        echo "  To install: sudo dnf install $1"
    fi
}
```

Explanation: This script uses if-then-else to detect the package manager and a nested if to check installation status. command -v tests whether a command exists. The case statement pattern-matches $PACKAGE against named strings; | lets multiple patterns share one branch. grep -E uses extended regular expressions to filter multiple fields in one pipe.

5.3 Script 3 — Disk and Permission Auditor

```
#!/bin/bash
# ── Array of directories to audit ──────────────────────────
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/var" "/usr/share")
# ── Display header ─────────────────────────────────────────
echo "================================================================"
echo "           DISK AND PERMISSION AUDITOR"
echo "           Auditing key Linux system directories"
echo "================================================================"
echo ""
printf "%-20s %-12s %-14s %-10s %s\n" "DIRECTORY" "SIZE" "PERMISSIONS" "OWNER" "GROUP"
echo "────────────────────────────────────────────────────────────────"
# ── For loop: iterate over each directory in the array ──────
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        LS_OUTPUT=$(ls -ld "$DIR" 2>/dev/null)
        PERMS=$(echo "$LS_OUTPUT" | awk '{print $1}')
        OWNER=$(echo "$LS_OUTPUT" | awk '{print $3}')
        GROUP=$(echo "$LS_OUTPUT" | awk '{print $4}')
        printf "%-20s %-12s %-14s %-10s %s\n" "$DIR" "$SIZE" "$PERMS" "$OWNER" "$GROUP"
    else
        printf "%-20s %s\n" "$DIR" "[ does not exist on this system ]"
    fi
done
echo "────────────────────────────────────────────────────────────────"
echo ""
# ── Section: Check Git-specific directories ────────────────┤
echo "================================================================"
echo "  GIT-SPECIFIC DIRECTORY CHECK"
echo "================================================================"
echo ""
# Common locations where Git installs its files on Linux
GIT_DIRS=("/usr/share/git-core" "/usr/lib/git-core" "/etc/gitconfig")
```

```
GIT_DIRS=("/usr/share/git-core" "/usr/lib/git-core" "/etc/gitconfig")
for GDIR in "${GIT_DIRS[@]}"; do
    if [ -e "$GDIR" ]; then
        LS_GIT=$(ls -ld "$GDIR" 2>/dev/null)
        GPERMS=$(echo "$LS_GIT" | awk '{print $1}')
        GOWNER=$(echo "$LS_GIT" | awk '{print $3}')
        echo "  Found: $GDIR"
        echo "    Permissions : $GPERMS"
        echo "    Owner       : $GOWNER"
    else
        echo "  Not found: $GDIR (Git may not be installed or path differs)"
    fi
    echo ""
done
# ── Section: Check current directory for a .git repository ──
echo "------------------------------------------------------------"
echo "  Checking for .git repository in current directory..."
echo "------------------------------------------------------------"
if [ -d ".git" ]; then
    echo ""
    echo "  A Git repository (.git/) exists in the current directory."
    GIT_SIZE=$(du -sh .git 2>/dev/null | cut -f1)S
    GIT_PERMS=$(ls -ld .git | awk '{print $1}')
    echo "  .git/ size        : $GIT_SIZE"
    echo "  .git/ permissions : $GIT_PERMS"
    echo ""
    if [ -f ".git/HEAD" ]; then
        echo "  Current HEAD: $(cat .git/HEAD)"
    fi
else
    echo ""
    echo "  No .git/ directory found here."
    echo "  Run from inside a Git repository to see repository-specific info."
```

Explanation: The central construct is a for loop over a Bash array. "${DIRS[@]}" expands all elements safely. du -sh reports disk usage in human-readable form; ls -ld lists directory metadata; awk '{print $N}' extracts specific fields. printf with format specifiers produces column-aligned output.

5.4 Script 4 — Log File Analyzer

```
done
# ── Display header ──────────────────────────────────────
echo ""
echo "================================================================"
echo "  LOG FILE ANALYZER"
echo "  File    : $LOGFILE"
echo "  Keyword : '$KEYWORD' (case-insensitive search)"
echo "================================================================"
echo ""
# ── Main loop: read file line by line ───────────────────
while IFS= read -r LINE; do
    TOTAL_LINES=$((TOTAL_LINES + 1))
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        MATCH_COUNT=$((MATCH_COUNT + 1))
    fi
done < "$LOGFILE"
# ── Print summary results ───────────────────────────────
echo "  SUMMARY"
echo "  ──────────────────────────────────────────────────────────"
echo "  Total lines in file      : $TOTAL_LINES"
echo "  Lines matching '$KEYWORD' : $MATCH_COUNT"
if [ "$TOTAL_LINES" -gt 0 ]; then
    PERCENT=$(( (MATCH_COUNT * 100) / TOTAL_LINES ))
    echo "  Match percentage         : ${PERCENT}%"
else
    echo "  The file appears to be empty."
fi
echo ""
echo "  LAST 5 MATCHING LINES"
echo "  ──────────────────────────────────────────────────────────"
LAST_MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" 2>/dev/null | tail -5)
if [ -z "$LAST_MATCHES" ]; then
    echo "  (No lines matching '$KEYWORD' were found in the file)"
else
```


```
else
    echo "  The file appears to be empty."
fi
echo ""
echo "  LAST 5 MATCHING LINES"
echo "  ──────────────────────────────────────────────────────────"
LAST_MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" 2>/dev/null | tail -5)
if [ -z "$LAST_MATCHES" ]; then
    echo "  (No lines matching '$KEYWORD' were found in the file)"
else
```


```
    echo "$LAST_MATCHES" | while IFS= read -r MATCH_LINE; do
        echo "  >> $MATCH_LINE"
    done
fi
echo ""
echo "================================================================"
echo "  End of Log File Analyzer"
echo "================================================================"
```

Explanation: The key construct is while IFS= read -r LINE — the correct Bash pattern for reading a file line by line. The file is fed via input redirection (< "$LOGFILE"), not via cat. Command-line arguments ($1, $2) and parameter expansion for defaults are demonstrated. A do-while style retry loop re-prompts for a valid file path. Integer arithmetic uses $((...)).

5.5 Script 5 — Open Source Manifesto Generator

```bash
#!/bin/bash
# ── Welcome banner ──────────────────────────────────────────
echo "================================================================"
echo "       THE OPEN SOURCE MANIFESTO GENERATOR"
echo "       A VITyarthi OSS Project Tool"
echo "       Author: Naba Krishna Hazarika | 24MIP10020"
echo "================================================================"
echo ""
echo "  Answer three questions to generate your personal open-source"
echo "  philosophy statement. Your answers will be composed into a"
echo "  paragraph and saved to a text file."
echo ""
echo "----------------------------------------------------------------"
echo ""
# ── Read user input interactively ──────────────────────────
read -r -p "  Q1. Name one open-source tool you use every day: " TOOL
while [ -z "$TOOL" ]; do
    echo "  Please type something — this question cannot be skipped."
    read -r -p "  Q1. Name one open-source tool you use every day: " TOOL
done
echo ""
read -r -p "  Q2. In one word, what does 'freedom' in software mean to you? " FREEDOM
while [ -z "$FREEDOM" ]; do
    echo "  Please enter at least one word."
    read -r -p "  Q2. In one word, what does 'freedom' mean to you? " FREEDOM
done

echo ""
read -r -p "  Q3. Name one thing you would build and share freely if you could: " BUILD
```

```bash
while [ -z "$BUILD" ]; do
    echo "  Please type your answer — even a small idea counts."
    read -r -p "  Q3. Name one thing you would build and share freely: " BUILD
done
echo ""
echo "----------------------------------------------------------------"
echo "  Composing your manifesto..."
echo "----------------------------------------------------------------"
echo ""

# ── Prepare variables ──────────────────────────────────────
DATE=$(date '+%d %B %Y')      # e.g., 29 March 2026
TIME=$(date '+%H:%M')         # e.g., 12:30
OUTPUT="manifesto_$(whoami).txt"
# ── Write the manifesto to a file ──────────────────────────
# Write the file header
echo "================================================================" > "$OUTPUT"
echo "  MY OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo "  By: Naba Krishna Hazarika (24MIP10020)" >> "$OUTPUT"
echo "  Generated: $DATE at $TIME" >> "$OUTPUT"
echo "================================================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"
# Write paragraph 1 — using $TOOL (string interpolation inside double quotes)
echo "  Every day I rely on $TOOL — a tool I did not have to buy," >> "$OUTPUT"
echo "  did not have to beg permission to use, and can inspect down" >> "$OUTPUT"
echo "  to its last line of code if I choose to. This is not a small" >> "$OUTPUT"
echo "  thing. It is the direct result of people choosing to build" >> "$OUTPUT"
echo "  in the open and share their work freely." >> "$OUTPUT"
echo "" >> "$OUTPUT"
# Write paragraph 2 — using $FREEDOM
echo "  To me, freedom in software means $FREEDOM. Not in an abstract" >> "$OUTPUT"
echo "  way, but in the practical sense: freedom to run, to study," >> "$OUTPUT"
```

```
echo "  way, but in the practical sense: freedom to run, to study," >> "$OUTPUT"
echo "  to modify,to explore things and to share. The four freedoms that the Free" >> "$OUTPUT"
echo "  Software Foundation articulates are not idealism. They are" >> "$OUTPUT"
echo "  the conditions under which knowledge compounds rather than" >> "$OUTPUT"
echo "  stagnates. Every open-source tool that exists is evidence" >> "$OUTPUT"
echo "  of what becomes possible when those conditions are met." >> "$OUTPUT"
echo "" >> "$OUTPUT"
# Write paragraph 3 — using $BUILD
echo "  If I could build one thing and share it freely, it would be" >> "$OUTPUT"
echo "  $BUILD. I would license it under GPL v2 — not because" >> "$OUTPUT"
echo "  it would make me famous, but because it is the surest way" >> "$OUTPUT"
echo "  to ensure that whatever I build remains available, forever," >> "$OUTPUT"
echo "  to everyone who might find it useful. Standing on the" >> "$OUTPUT"
echo "  shoulders of giants means, eventually, being someone else's" >> "$OUTPUT"
echo "  giant. This is my commitment to that chain." >> "$OUTPUT"
echo "" >> "$OUTPUT"
# Write footer
echo "----------------------------------------------------------------" >> "$OUTPUT"
echo "  Course: Open Source Software (NGMC) | VITyarthi 2026" >> "$OUTPUT"
echo "================================================================" >> "$OUTPUT"
# ── Confirm save and display the manifesto ────────────────────
echo "  Manifesto successfully saved to: $OUTPUT"
echo ""
echo "================================================================"
echo "  YOUR GENERATED MANIFESTO"
echo "================================================================"
echo ""
# cat reads the file and prints its contents to the terminal
cat "$OUTPUT"
echo ""
echo "================================================================"
echo "  End of Manifesto Generator"
echo "================================================================"
```

Explanation: Script 5 uses read -r -p for interactive input. Input validation uses while [-z "$VAR"] to re-prompt until the user answers. String concatenation in Bash needs no operator — placing a variable inside a double-quoted string interpolates it. File writing uses > to create the file and >> to append subsequent lines. cat displays the saved file on screen after writing.

6. Conclusion

Git started as a crisis response and became infrastructure that modern software development cannot exist without. What this audit has tried to show is that this outcome was not accidental — it was a direct result of the licence choice and the design philosophy. Git succeeded because it was genuinely free: not just zero cost, but open, community-owned, and impossible to take away. The BitKeeper story is the clearest possible proof of why that matters. A tool that tens of thousands of people depended on disappeared overnight because one company changed its mind. That cannot happen to Git.

The Linux footprint analysis showed how cleanly Git integrates with the operating system — no bloated service, no special permissions, just files and a well-designed command-line interface built on standard Unix tools. The ecosystem section showed what becomes possible when a well-designed tool is truly open: hundreds of platforms, clients, and workflows built on top of it without anyone asking permission. The comparison with Perforce was honest: there are specific use cases where proprietary software has real advantages. But for the vast majority of software projects, Git is the right answer, and the fact that it is free and community-governed only strengthens that.

The five shell scripts in this project are more than assessment tasks. Writing and running them meant working directly in the Linux environment that Git was built to serve — checking permissions, reading directories, parsing log files, and writing output to files. That practical familiarity with the tools underneath is the foundation that the rest of a software career is built on.

References

1. GNU Project. The Free Software Definition. https://www.gnu.org/philosophy/free-sw.html
2. Open Source Initiative. The Open Source Definition. https://opensource.org/osd
3. Raymond, E.S. The Cathedral and the Bazaar. http://catb.org/~esr/writings/cathedral-bazaar/
4. Git Source Repository and COPYING file. https://github.com/git/git
5. Software Freedom Conservancy. https://sfconservancy.org/
6. Git Reference Manual. https://git-scm.com/docs
7. SPDX Licence List. https://spdx.org/licenses/
8. Shotts, W. The Linux Command Line. https://linuxcommand.org
9. GNU Bash Manual. https://www.gnu.org/software/bash/manual/